

TP 8 : Apprentissage non supervisé

Compléments de Mathématiques 2 - L3 MIASHS

HAMLIL Mohamed

2026-05-19

Table des matières

1	0. Jeu de données	1
1.1	0.1 Charger le jeu de données	1
1.2	0.2 Caractéristiques du jeu de données	2
2	1. K-moyennes (K-means)	3
2.1	1.0 Pré-processing	3
2.2	1.1 Pour une valeur de K fixée	3
2.2.1	1.1.1 La fonction <code>kmeans</code>	3
2.2.2	1.1.2 Coefficient de silhouette et indice de Davies Bouldin	8
2.3	1.2 Choisir la valeur de K	13
2.4	1.3 K-médoïdes	15
3	2. Clustering hiérarchique	16
3.1	2.1 La fonction <code>hclust</code>	16
3.2	2.2 Critère d'arrêt	17
3.3	2.3 Impact du cluster linkage sur la forme des clusters	18
3.4	2.4 Coefficient de silhouette et indice de Davies Bouldin	22
4	3. Interprétation des clusters	23

1 0. Jeu de données

1.1 0.1 Charger le jeu de données

Charger le jeu de données `iris` du package `datasets`

```
library(datasets)
data <- iris[, -5]
```

Note

La cinquième colonne du jeu de données contient la variable **Species** que nous n'utiliserons pas.

1.2 0.2 Caractéristiques du jeu de données

Donner le nombre de lignes et le nombre de colonnes du data frame :

```
nrow(data)
```

```
[1] 150
```

```
ncol(data)
```

```
[1] 4
```

```
dim(data)
```

```
[1] 150 4
```

Il y a 150 lignes (observations) et 4 colonnes (variables).

Lister le nom de chaque variable :

```
names(data)
```

```
[1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"
```

Description de chacune des variables :

- **Sepal.Length** : longueur des sépales en cm
- **Sepal.Width** : largeur des sépales en cm
- **Petal.Length** : longueur des pétales en cm
- **Petal.Width** : largeur des pétales en cm

Les 4 variables sont toutes quantitatives continues (mesures en cm). La variable **Species** (retirée) était qualitative nominale.

```
str(data)
```

```
'data.frame': 150 obs. of 4 variables:
 $ Sepal.Length: num 5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num 3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num 1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num 0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
```

Les 4 variables sont de type **numeric** (double), ce qui est cohérent avec leur nature quantitative continue. Pas de changement nécessaire.

2 1. K-moyennes (K-means)

2.1 1.0 Pré-processing

```
data_stand <- as.data.frame(scale(data))
```

La fonction `scale()` centre et réduit (standardise) chaque variable : elle soustrait la moyenne et divise par l'écart-type. On obtient ainsi des variables de moyenne 0 et d'écart-type 1. C'est nécessaire avant d'appliquer K-means car l'algorithme utilise la distance euclidienne : si les variables ne sont pas sur la même échelle, celles avec les plus grandes valeurs domineraient le calcul de distance.

2.2 1.1 Pour une valeur de K fixée

2.2.1 1.1.1 La fonction `kmeans`

```
set.seed(1357)
kmeans_res <- kmeans(x = data_stand, centers = 3)
kmeans_clusters <- as.factor(kmeans_res$cluster)
kmeans_centroids <- as.data.frame(kmeans_res$centers)
kmeans_clusters
```

```

[1] 3 2 2 2 3 3 3 3 2 2 3 3 2 2 3 3 3 3 3 3 3 3 2 3 3 3 2 2 3 3 3 2 2 3
[38] 3 2 3 3 2 2 3 3 2 3 2 3 3 1 1 1 1 1 1 2 1 1 2 1 1 1 1 1 1 1 1 1 1
[75] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1
[112] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
[149] 1 1
Levels: 1 2 3

```

```
kmeans_centroids
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
1	0.5690971	-0.3705265	0.6888118	0.6609378
2	-1.3232208	-0.3718921	-1.1334386	-1.1111395
3	-0.8135055	1.3145538	-1.2825372	-1.2156393

Ce code applique l'algorithme K-means avec K=3 clusters sur les données standardisées. `set.seed(1357)` fixe la graine aléatoire pour la reproductibilité. `kmeans_res$cluster` contient l'affectation de chaque observation à un cluster (converti en factor pour le graphique). `kmeans_res$centers` contient les coordonnées des 3 centroïdes finaux.

Lorsque `centers` est un entier (ici 3), les centroïdes initiaux sont choisis aléatoirement parmi les observations du jeu de données (méthode par défaut de Hartigan-Wong).

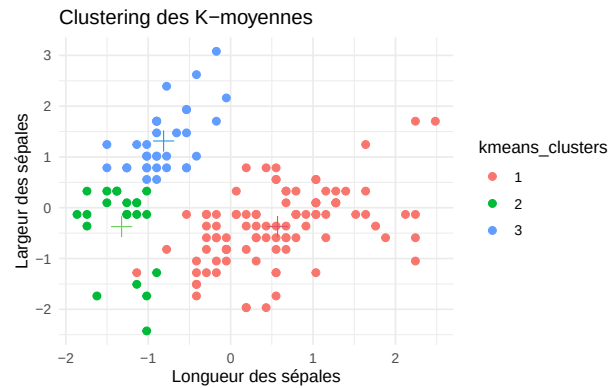
La composante `kmeans_res$tot.withinss` contient la WCSS totale (somme globale). `kmeans_res$withinss` contient la somme des carrés intra-cluster pour chaque cluster individuellement.

```
library(ggplot2)
```

```

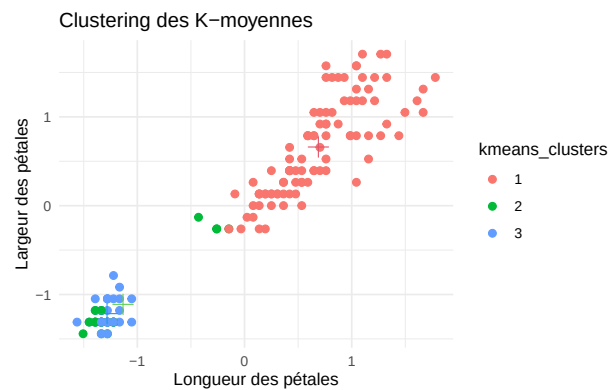
ggplot(data_stand, aes(x = Sepal.Length, y = Sepal.Width,
                      color = kmeans_clusters)) +
  geom_point(size = 2) +
  geom_point(data = kmeans_centroids[, 1:2], aes(x = Sepal.Length,
                                                y = Sepal.Width),
            color = 2:4, size = 4, shape = 3) +
  labs(title = "Clustering des K-moyennes", x = "Longueur des sépales",
       y = "Largeur des sépales") +
  theme_minimal()

```



Même graphe pour la longueur et la largeur des pétales :

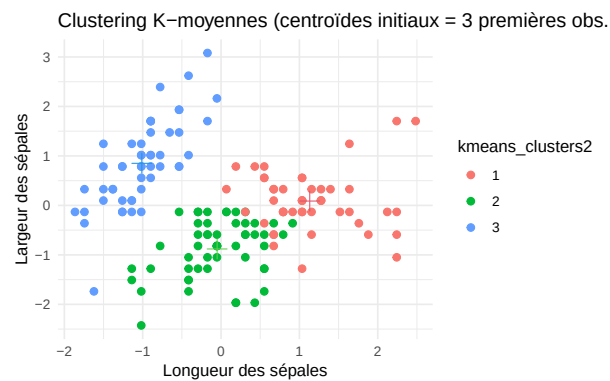
```
ggplot(data_stand, aes(x = Petal.Length, y = Petal.Width,
                      color = kmeans_clusters)) +
  geom_point(size = 2) +
  geom_point(data = kmeans_centroids[, 3:4], aes(x = Petal.Length,
                                                  y = Petal.Width),
            color = 2:4, size = 4, shape = 3) +
  labs(title = "Clustering des K-moyennes", x = "Longueur des pétales",
       y = "Largeur des pétales") +
  theme_minimal()
```



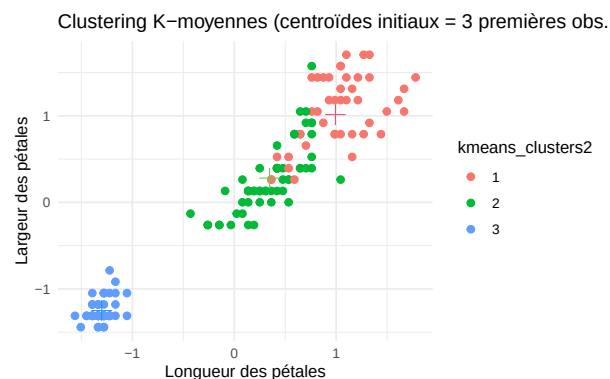
Utilisation des 3 premières instances comme centroïdes initiaux :

```
set.seed(1357)
kmeans_res2      <- kmeans(x = data_stand, centers = data_stand[1:3, ])
kmeans_clusters2 <- as.factor(kmeans_res2$cluster)
kmeans_centroids2 <- as.data.frame(kmeans_res2$centers)
```

```
ggplot(data_stand, aes(x = Sepal.Length, y = Sepal.Width,
                      color = kmeans_clusters2)) +
  geom_point(size = 2) +
  geom_point(data = kmeans_centroids2[, 1:2], aes(x = Sepal.Length,
                                                  y = Sepal.Width),
            color = 2:4, size = 4, shape = 3) +
  labs(title = "Clustering K-moyennes (centroïdes initiaux = 3 premières obs.)",
       x = "Longueur des sépales", y = "Largeur des sépales") +
  theme_minimal()
```



```
ggplot(data_stand, aes(x = Petal.Length, y = Petal.Width,
                      color = kmeans_clusters2)) +
  geom_point(size = 2) +
  geom_point(data = kmeans_centroids2[, 3:4], aes(x = Petal.Length,
                                                  y = Petal.Width),
            color = 2:4, size = 4, shape = 3) +
  labs(title = "Clustering K-moyennes (centroïdes initiaux = 3 premières obs.)",
       x = "Longueur des pétales", y = "Largeur des pétales") +
  theme_minimal()
```

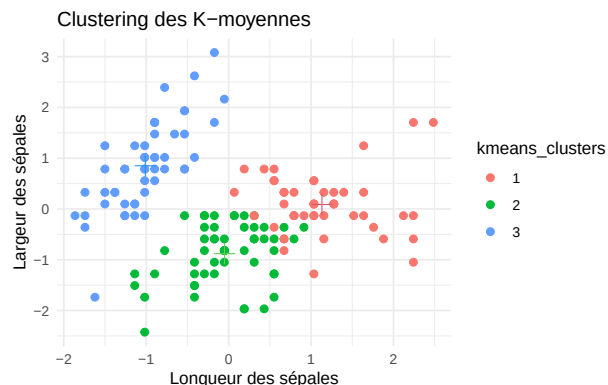


Les résultats du clustering peuvent différer selon le choix des centroïdes initiaux. Les 3 premières observations appartiennent toutes à l'espèce *setosa*, donc les centroïdes initiaux sont très proches les uns des autres, ce qui peut conduire à un clustering différent (potentiellement sous-optimal) par rapport à une initialisation aléatoire.

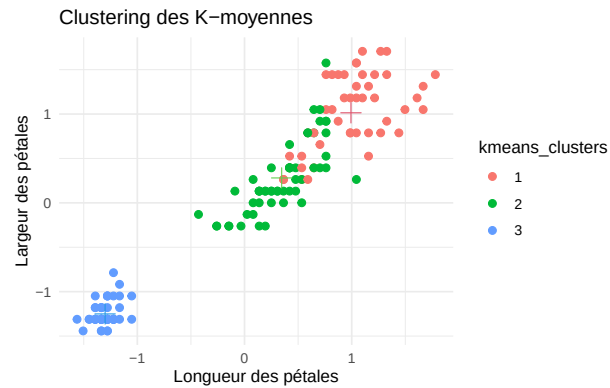
L'argument `nstart` permet de relancer l'algorithme K-means plusieurs fois avec des centroïdes initiaux différents (choisis aléatoirement), puis de conserver le meilleur résultat (celui avec la plus faible WCSS totale). Cela réduit le risque de converger vers un minimum local.

```
set.seed(1357)
kmeans_res      <- kmeans(x = data_stand, centers = 3, nstart = 20)
kmeans_clusters <- as.factor(kmeans_res$cluster)
kmeans_centroids <- as.data.frame(kmeans_res$centers)

ggplot(data_stand, aes(x = Sepal.Length, y = Sepal.Width, color = kmeans_clusters)) +
  geom_point(size = 2) +
  geom_point(data = kmeans_centroids[, 1:2], aes(x = Sepal.Length, y = Sepal.Width),
            color = 2:4, size = 4, shape = 3) +
  labs(title = "Clustering des K-moyennes", x = "Longueur des sépales",
       y = "Largeur des sépales") +
  theme_minimal()
```



```
ggplot(data_stand, aes(x = Petal.Length, y = Petal.Width, color = kmeans_clusters)) +
  geom_point(size = 2) +
  geom_point(data = kmeans_centroids[, 3:4], aes(x = Petal.Length, y = Petal.Width),
            color = 2:4, size = 4, shape = 3) +
  labs(title = "Clustering des K-moyennes", x = "Longueur des pétales",
       y = "Largeur des pétales") +
  theme_minimal()
```



i Note

Le cluster vert n'apparaît pas clairement séparé des deux autres.

2.2.2 1.1.2 Coefficient de silhouette et indice de Davies Bouldin

```
dist(data_stand[1:5, ], upper = TRUE, diag = TRUE)
```

	1	2	3	4	5
1	0.0000000	1.1722914	0.8427840	1.0999999	0.2592702
2	1.1722914	0.0000000	0.5216255	0.4325508	1.3818560
3	0.8427840	0.5216255	0.0000000	0.2829432	0.9882608
4	1.0999999	0.4325508	0.2829432	0.0000000	1.2459861
5	0.2592702	1.3818560	0.9882608	1.2459861	0.0000000

Cette ligne calcule la matrice des distances euclidiennes entre les 5 premières observations du jeu de données standardisé. `upper = TRUE` affiche la partie triangulaire supérieure, et `diag = TRUE` affiche la diagonale (qui vaut 0).

```
library(cluster)
```

```
kmeans_sil <- silhouette(x = kmeans_res$cluster, dist = dist(data_stand))
kmeans_sil
```

	cluster	neighbor	sil_width
[1,]	3	2	0.73419485
[2,]	3	2	0.56827391

[3,]	3	2	0.67754724
[4,]	3	2	0.62050159
[5,]	3	2	0.72847412
[6,]	3	2	0.60988485
[7,]	3	2	0.69838355
[8,]	3	2	0.73081691
[9,]	3	2	0.48821004
[10,]	3	2	0.63154089
[11,]	3	2	0.67418286
[12,]	3	2	0.72179392
[13,]	3	2	0.57847213
[14,]	3	2	0.54944562
[15,]	3	2	0.55294537
[16,]	3	2	0.45806886
[17,]	3	2	0.62112236
[18,]	3	2	0.72875221
[19,]	3	2	0.58590385
[20,]	3	2	0.67746392
[21,]	3	2	0.66042913
[22,]	3	2	0.69021224
[23,]	3	2	0.69653476
[24,]	3	2	0.63737612
[25,]	3	2	0.70132024
[26,]	3	2	0.54494792
[27,]	3	2	0.70471457
[28,]	3	2	0.72151031
[29,]	3	2	0.71451919
[30,]	3	2	0.66776336
[31,]	3	2	0.62562482
[32,]	3	2	0.64783099
[33,]	3	2	0.58967022
[34,]	3	2	0.54088124
[35,]	3	2	0.62843507
[36,]	3	2	0.67614681
[37,]	3	2	0.66543365
[38,]	3	2	0.72441906
[39,]	3	2	0.55262190
[40,]	3	2	0.72470474
[41,]	3	2	0.73333453
[42,]	3	2	0.07766666
[43,]	3	2	0.63725809
[44,]	3	2	0.66371521
[45,]	3	2	0.64863547

[46,]	3	2	0.55934650
[47,]	3	2	0.67881722
[48,]	3	2	0.66606206
[49,]	3	2	0.68912398
[50,]	3	2	0.71077363
[51,]	1	2	0.34169992
[52,]	1	2	0.16627565
[53,]	1	2	0.35854546
[54,]	2	1	0.54373992
[55,]	2	1	0.13270610
[56,]	2	1	0.53878741
[57,]	1	2	0.23206569
[58,]	2	3	0.40978046
[59,]	2	1	0.03768897
[60,]	2	1	0.54390267
[61,]	2	3	0.42467141
[62,]	2	1	0.34088756
[63,]	2	1	0.46946550
[64,]	2	1	0.33187646
[65,]	2	1	0.49517920
[66,]	1	2	0.18906699
[67,]	2	1	0.39249659
[68,]	2	1	0.54918014
[69,]	2	1	0.40350566
[70,]	2	1	0.58259179
[71,]	1	2	0.03645185
[72,]	2	1	0.45853055
[73,]	2	1	0.36638600
[74,]	2	1	0.42610282
[75,]	2	1	0.21912232
[76,]	1	2	0.05112346
[77,]	1	2	0.04063911
[78,]	1	2	0.34222491
[79,]	2	1	0.37447730
[80,]	2	1	0.55591003
[81,]	2	1	0.56667628
[82,]	2	1	0.55697725
[83,]	2	1	0.56859072
[84,]	2	1	0.39023598
[85,]	2	1	0.40497788
[86,]	1	2	0.10973108
[87,]	1	2	0.27953553
[88,]	2	1	0.41829609

[89,]	2	1	0.44045285
[90,]	2	1	0.58629734
[91,]	2	1	0.58416837
[92,]	2	1	0.24125752
[93,]	2	1	0.58008127
[94,]	2	3	0.43896605
[95,]	2	1	0.58397553
[96,]	2	1	0.43067583
[97,]	2	1	0.50035119
[98,]	2	1	0.33948876
[99,]	2	3	0.43311107
[100,]	2	1	0.55292391
[101,]	1	2	0.42463909
[102,]	2	1	0.34345454
[103,]	1	2	0.52207700
[104,]	1	2	0.15132973
[105,]	1	2	0.43894193
[106,]	1	2	0.47218801
[107,]	2	1	0.47007943
[108,]	1	2	0.44360797
[109,]	1	2	0.01675919
[110,]	1	2	0.46637587
[111,]	1	2	0.44615652
[112,]	1	2	-0.01058434
[113,]	1	2	0.50147349
[114,]	2	1	0.37966719
[115,]	2	1	0.05778708
[116,]	1	2	0.44552087
[117,]	1	2	0.35179075
[118,]	1	2	0.41131991
[119,]	1	2	0.33087257
[120,]	2	1	0.41625797
[121,]	1	2	0.54976242
[122,]	2	1	0.31780905
[123,]	1	2	0.40083606
[124,]	2	1	0.17744927
[125,]	1	2	0.52568825
[126,]	1	2	0.51433621
[127,]	2	1	0.17430033
[128,]	1	2	-0.02489394
[129,]	1	2	0.22253544
[130,]	1	2	0.43638496
[131,]	1	2	0.41286324

```

[132,]      1      2 0.39874279
[133,]      1      2 0.24284231
[134,]      2      1 0.19544119
[135,]      2      1 0.34505079
[136,]      1      2 0.46884592
[137,]      1      2 0.42243015
[138,]      1      2 0.36093602
[139,]      2      1 0.08594176
[140,]      1      2 0.53818328
[141,]      1      2 0.50861824
[142,]      1      2 0.51142970
[143,]      2      1 0.34345454
[144,]      1      2 0.54298088
[145,]      1      2 0.50444897
[146,]      1      2 0.46228025
[147,]      2      1 0.22992489
[148,]      1      2 0.38121152
[149,]      1      2 0.38714414
[150,]      2      1 0.09788140
attr(,"Ordered")
[1] FALSE
attr(,"call")
silhouette.default(x = kmeans_res$cluster, dist = dist(data_stand))
attr(,"class")
[1] "silhouette"

```

La fonction `silhouette()` retourne une matrice avec, pour chaque observation : le cluster auquel elle appartient, le cluster voisin le plus proche, et son coefficient de silhouette individuel (entre -1 et 1). Une valeur proche de 1 indique que l'observation est bien assignée à son cluster.

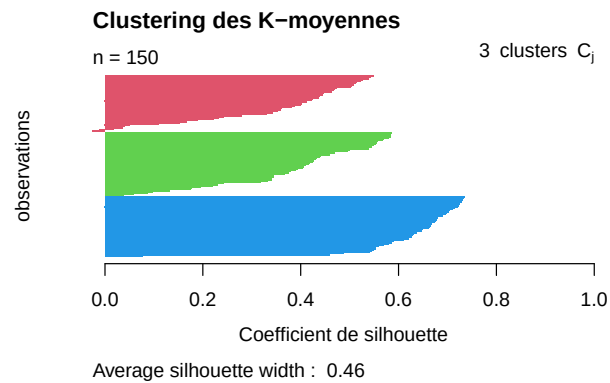
Coefficient de silhouette moyen :

```
mean(kmeans_sil[, 3])
```

```
[1] 0.4599482
```

```
plot(kmeans_sil, col = 2:4, do.clus.stat = FALSE, main = "Clustering des K-moyennes",
     xlab = "Coefficient de silhouette", ylab = "observations")

```



Ce code trace le graphique des silhouettes : pour chaque cluster, les observations sont triées par coefficient de silhouette décroissant. Cela permet de visualiser la qualité de l'affectation de chaque observation et d'identifier les observations mal classées (silhouette négative).

```
library(clusterSim)
```

Indice de Davies-Bouldin :

```
db_res <- index.DB(x = data_stand, cl = kmeans_res$cluster)
db_res$DB
```

```
[1] 0.9140889
```

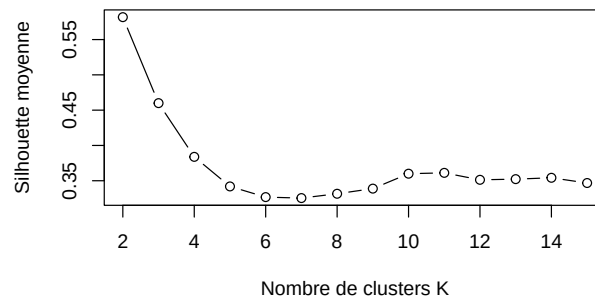
2.3 1.2 Choisir la valeur de K

```
kmeans_silhouette <- NULL
kmeans_wcss <- NULL
for (k in 2:15) {
  kmeans_res <- kmeans(x = data_stand, centers = k, nstart = 20)

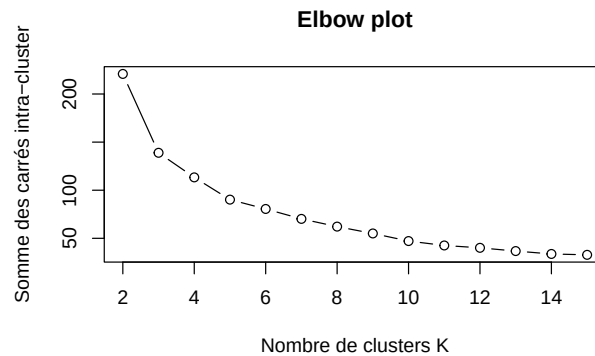
  # coefficient de silhouette
  kmeans_sil <- silhouette(x = kmeans_res$cluster, dist = dist(data_stand))
  kmeans_silhouette <- c(kmeans_silhouette, mean(kmeans_sil[, 3]))

  # somme des carrés intra-cluster (wcss)
  kmeans_wcss <- c(kmeans_wcss, kmeans_res$tot.withinss)
}
```

```
plot(2:15, kmeans_silhouette, type = "b", xlab = "Nombre de clusters K",
     ylab = "Silhouette moyenne")
```



```
plot(2:15, kmeans_wcss, type = "b", xlab = "Nombre de clusters K",
     ylab = "Somme des carrés intra-cluster", main = "Elbow plot")
```



D'après le graphique de la silhouette moyenne, $K=2$ donne la silhouette la plus élevée. D'après l'elbow plot, on observe un "coude" autour de $K=2$ ou $K=3$. Le choix $K=2$ maximise la silhouette, mais $K=3$ est aussi un bon choix car il correspond au nombre réel d'espèces dans le jeu de données iris.

Clustering avec $K=2$:

```
set.seed(1357)
kmeans_res_k2 <- kmeans(x = data_stand, centers = 2, nstart = 20)
kmeans_sil_k2 <- silhouette(x = kmeans_res_k2$cluster, dist = dist(data_stand))

# Silhouette moyenne
cat("Silhouette moyenne (K=2) :", mean(kmeans_sil_k2[, 3]), "\n")
```

Silhouette moyenne (K=2) : 0.58175

```
# Indice de Davies-Bouldin
db_k2 <- index.DB(x = data_stand, cl = kmeans_res_k2$cluster)
cat("Indice de Davies-Bouldin (K=2) :", db_k2$DB, "\n")
```

Indice de Davies-Bouldin (K=2) : 0.6827699

2.4 1.3 K-médoïdes

```
set.seed(1357)
kmedoids_res <- pam(x = data_stand, k = 3)
kmedoids_clusters <- as.factor(kmedoids_res$clustering)
kmedoids_medoids <- as.data.frame(kmedoids_res$medoids)
kmedoids_clusters
```

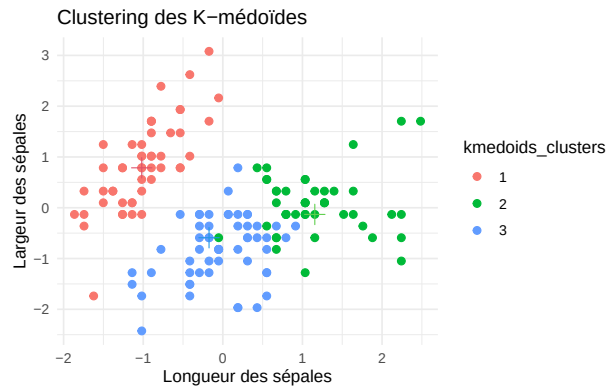
```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
[38] 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 3 3 3 2 3 3 3 3 3 3 3 2 3 3 3 3 3 3
[75] 3 2 2 2 3 3 3 3 3 3 3 3 2 3 3 3 3 3 3 3 3 3 3 3 2 3 2 2 2 2 3 2 2 2
[112] 2 2 3 2 2 2 2 2 3 2 3 2 3 2 2 3 3 2 2 2 2 2 3 3 2 2 2 3 2 2 2 3 2
[149] 2 3
Levels: 1 2 3
```

```
kmedoids_medoids
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
1	-1.0184372	0.7861738	-1.2791040	-1.3110521
2	1.1553023	-0.1315388	0.9868021	1.1816087
3	-0.1730941	-0.5903951	0.4203256	0.1320673

Tracer les clusters par rapport à la longueur et la largeur des sépales, avec les médoïdes :

```
ggplot(data_stand, aes(x = Sepal.Length, y = Sepal.Width,
                       color = kmedoids_clusters)) +
  geom_point(size = 2) +
  geom_point(data = kmedoids_medoids[, 1:2], aes(x = Sepal.Length,
                                                y = Sepal.Width),
            color = 2:4, size = 4, shape = 3) +
  labs(title = "Clustering des K-médoïdes", x = "Longueur des sépales",
       y = "Largeur des sépales") +
  theme_minimal()
```



Les résultats sont similaires à ceux de K-means, mais les médoïdes sont des observations réelles du jeu de données (contrairement aux centroïdes de K-means qui sont des moyennes). La méthode des K-médoïdes est plus robuste aux outliers car elle utilise des points existants comme centres.

3 2. Clustering hiérarchique

3.1 2.1 La fonction `hclust`

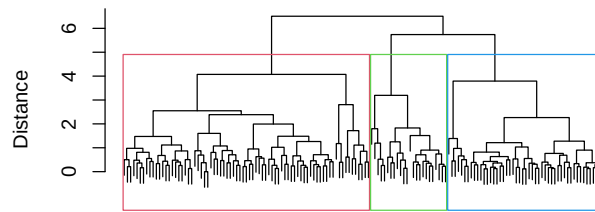
Il est nécessaire de standardiser les variables pour la même raison que pour K-means : le clustering hiérarchique repose sur le calcul de distances entre observations. Si les variables ne sont pas standardisées, celles avec les plus grandes valeurs auront plus de poids.

La fonction `hclust` fait du clustering ascendant hiérarchique (agglomératif). Elle part de chaque observation comme un cluster individuel, puis fusionne les clusters les plus proches à chaque étape, jusqu'à n'avoir plus qu'un seul cluster.

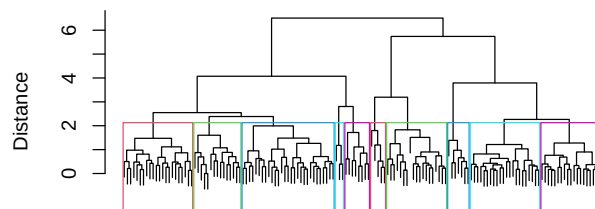
```
set.seed(1357)
hclust_res <- hclust(d = dist(data_stand), method = "complete")
```

Liens de dissimilarité disponibles : `method = "single"` pour le lien minimum, `method = "average"` pour le lien moyen, et `method = "ward.D2"` pour le lien de Ward.

```
plot(hclust_res, main = "Dendrogramme du clustering hiérarchique", xlab = "",
     sub = "", ylab = "Distance")
```


Dendrogramme du clustering hiérarchique

```
plot(hclust_res, main = "Dendrogramme du clustering hiérarchique", xlab = "",
     sub = "", ylab = "Distance", labels = FALSE)
rect.hclust(tree = hclust_res, h = 2, border = c(2:6, 2:6))
```

Dendrogramme du clustering hiérarchique

3.3 2.3 Impact du cluster linkage sur la forme des clusters

```
library(gridExtra)
```

```
p1 <- ggplot(data_stand, aes(x = Sepal.Length, y = Sepal.Width,
                             color = as.factor(cutree(hclust_res, k = 10)))) +
  geom_point(size = 1) +
  labs(title = "Clustering hiérarchique", x = "", y = "Largeur des sépales") +
  guides(fill = "none", color = "none", linetype = "none", shape = "none") +
  theme_minimal()
```

```
p2 <- ggplot(data_stand, aes(x = Sepal.Length, y = Sepal.Width,
                             color = as.factor(cutree(hclust_res, k = 9)))) +
  geom_point(size = 1) +
  labs(title = "", x = "", y = "") +
  guides(fill = "none", color = "none", linetype = "none", shape = "none") +
  theme_minimal()
```

```
p3 <- ggplot(data_stand, aes(x = Sepal.Length, y = Sepal.Width,
```

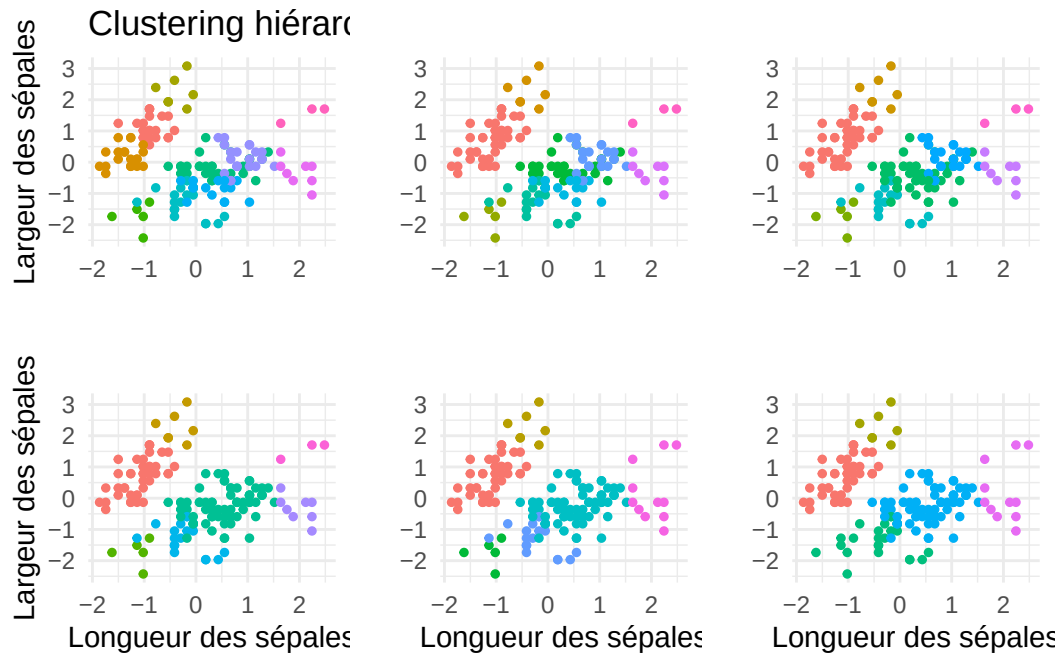
```
                                color = as.factor(cutree(hclust_res, k = 8)))) +
geom_point(size = 1) +
labs(title = "", x = "", y = "") +
guides(fill = "none", color = "none", linetype = "none", shape = "none") +
theme_minimal()

p4 <- ggplot(data_stand, aes(x = Sepal.Length, y = Sepal.Width,
                                color = as.factor(cutree(hclust_res, k = 7)))) +
geom_point(size = 1) +
labs(title = "", x = "Longueur des sépales", y = "Largeur des sépales") +
guides(fill = "none", color = "none", linetype = "none", shape = "none") +
theme_minimal()

p5 <- ggplot(data_stand, aes(x = Sepal.Length, y = Sepal.Width,
                                color = as.factor(cutree(hclust_res, k = 6)))) +
geom_point(size = 1) +
labs(title = "", x = "Longueur des sépales", y = "") +
guides(fill = "none", color = "none", linetype = "none", shape = "none") +
theme_minimal()

p6 <- ggplot(data_stand, aes(x = Sepal.Length, y = Sepal.Width,
                                color = as.factor(cutree(hclust_res, k = 5)))) +
geom_point(size = 1) +
labs(title = "", x = "Longueur des sépales", y = "") +
guides(fill = "none", color = "none", linetype = "none", shape = "none") +
theme_minimal()

grid.arrange(p1, p2, p3, p4, p5, p6, ncol = 3, nrow = 2)
```



Pour un même nombre de clusters, la forme est impactée par le cluster linkage utilisé :

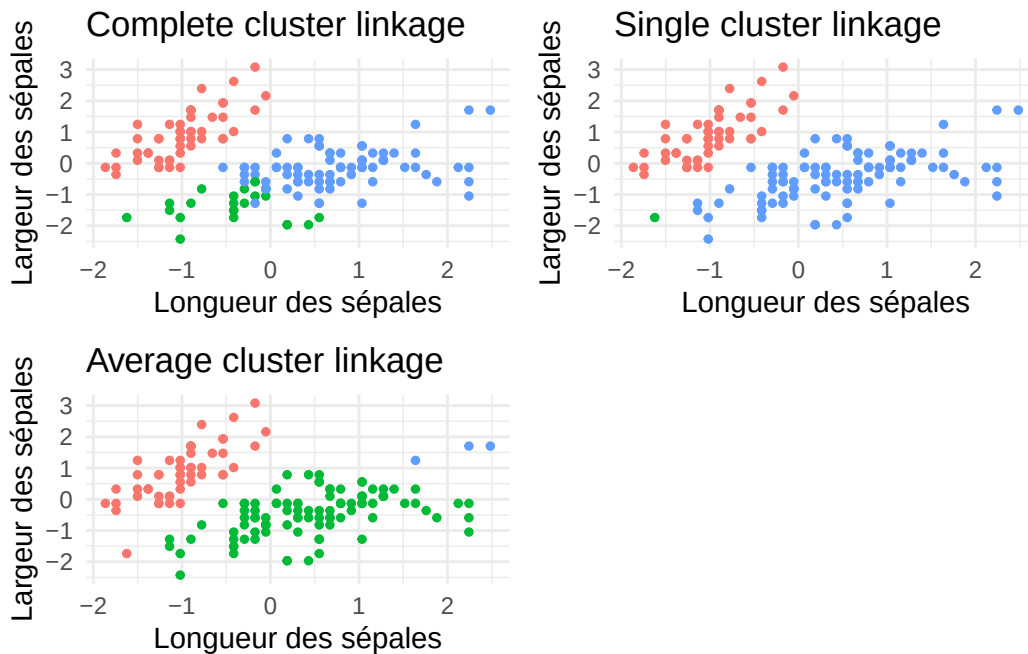
```
hclust_res1 <- hclust(dist(data_stand), method = "complete")
p1 <- ggplot(data_stand, aes(x = Sepal.Length, y = Sepal.Width,
                             color = as.factor(cutree(hclust_res1, k = 3)))) +
  geom_point(size = 1) +
  labs(title = "Complete cluster linkage", x = "Longueur des sépales",
       y = "Largeur des sépales") +
  guides(fill = "none", color = "none", linetype = "none", shape = "none") +
  theme_minimal()

hclust_res2 <- hclust(dist(data_stand), method = "single")
p2 <- ggplot(data_stand, aes(x = Sepal.Length, y = Sepal.Width,
                             color = as.factor(cutree(hclust_res2, k = 3)))) +
  geom_point(size = 1) +
  labs(title = "Single cluster linkage", x = "Longueur des sépales",
       y = "Largeur des sépales") +
  guides(fill = "none", color = "none", linetype = "none", shape = "none") +
  theme_minimal()

hclust_res3 <- hclust(dist(data_stand), method = "average")
p3 <- ggplot(data_stand, aes(x = Sepal.Length, y = Sepal.Width,
                             color = as.factor(cutree(hclust_res3, k = 3)))) +
```

```
geom_point(size = 1) +
labs(title = "Average cluster linkage", x = "Longueur des sépales",
      y = "Largeur des sépales") +
guides(fill = "none", color = "none", linetype = "none", shape = "none") +
theme_minimal()

grid.arrange(p1, p2, p3, ncol = 2, nrow = 2)
```



```
hclust_res1 <- hclust(dist(data_stand), method = "complete")
p1 <- ggplot(data_stand, aes(x = Petal.Length, y = Petal.Width,
                             color = as.factor(cutree(hclust_res1, k = 3)))) +
  geom_point(size = 1) +
  labs(title = "Complete cluster linkage", x = "Longueur des pétales",
        y = "Largeur des pétales") +
  guides(fill = "none", color = "none", linetype = "none", shape = "none") +
  theme_minimal()

hclust_res2 <- hclust(dist(data_stand), method = "single")
p2 <- ggplot(data_stand, aes(x = Petal.Length, y = Petal.Width,
                             color = as.factor(cutree(hclust_res2, k = 3)))) +
  geom_point(size = 1) +
  labs(title = "Single cluster linkage", x = "Longueur des pétales",
        y = "Largeur des pétales") +
```

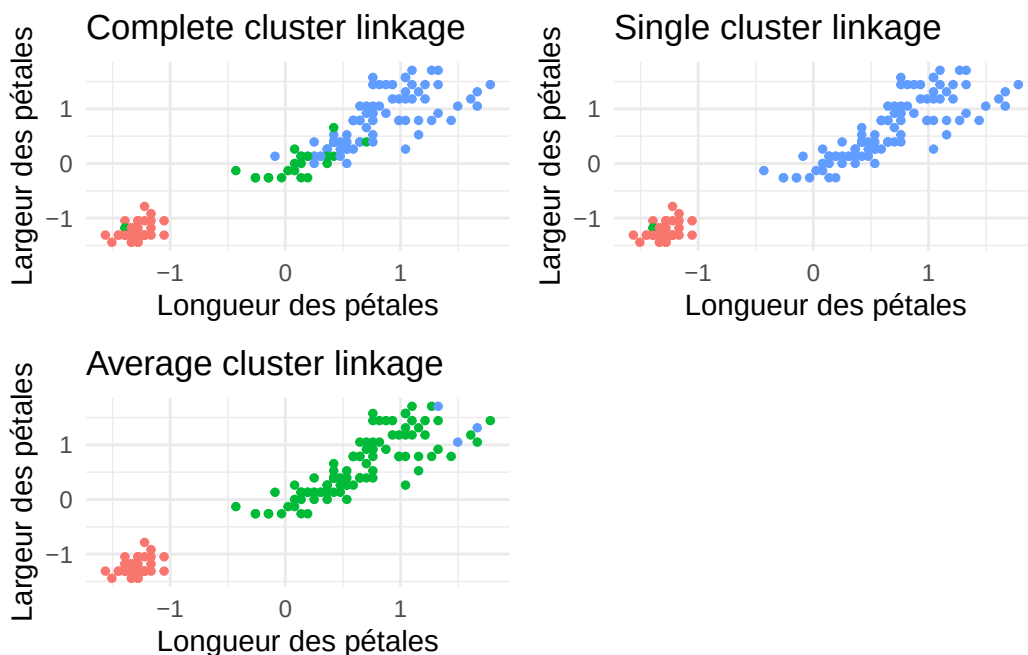
```

guides(fill = "none", color = "none", linetype = "none", shape = "none") +
theme_minimal()

hclust_res3 <- hclust(dist(data_stand), method = "average")
p3 <- ggplot(data_stand, aes(x = Petal.Length, y = Petal.Width,
                           color = as.factor(cutree(hclust_res3, k = 3)))) +
  geom_point(size = 1) +
  labs(title = "Average cluster linkage", x = "Longueur des pétales",
       y = "Largeur des pétales") +
  guides(fill = "none", color = "none", linetype = "none", shape = "none") +
  theme_minimal()

grid.arrange(p1, p2, p3, ncol = 2, nrow = 2)

```

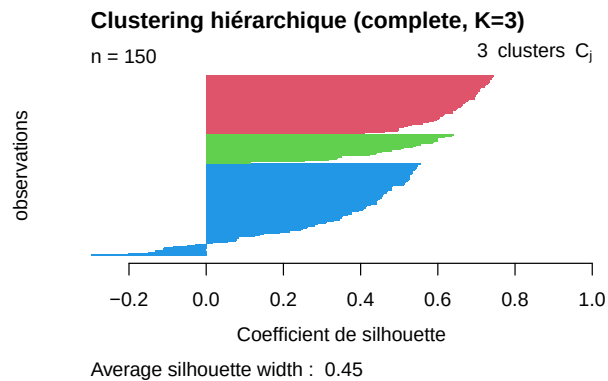


3.4 2.4 Coefficient de silhouette et indice de Davies Bouldin

```

hclust_clusters <- cutree(hclust_res, k = 3)
hclust_sil <- silhouette(x = hclust_clusters, dist = dist(data_stand))
plot(hclust_sil, col = 2:4, do.clus.stat = FALSE,
     main = "Clustering hiérarchique (complete, K=3)",
     xlab = "Coefficient de silhouette", ylab = "observations")

```



```
cat("Silhouette moyenne :", mean(hclust_sil[, 3]), "\n")
```

Silhouette moyenne : 0.4496185

Indice de Davies-Bouldin :

```
db_hclust <- index.DB(x = data_stand, cl = hclust_clusters)
cat("Indice de Davies-Bouldin (clustering hiérarchique, K=3) :", db_hclust$DB, "\n")
```

Indice de Davies-Bouldin (clustering hiérarchique, K=3) : 0.8580894

On pourrait aussi utiliser le coefficient de silhouette moyen pour choisir le meilleur nombre de clusters.

4 3. Interprétation des clusters

Les clusters obtenus correspondent assez bien aux 3 espèces d'iris du jeu de données (setosa, versicolor, virginica). Le cluster 1 (setosa) est très bien séparé des deux autres, avec des pétales beaucoup plus petits. Les clusters 2 (versicolor) et 3 (virginica) sont plus difficiles à distinguer car ces deux espèces ont des caractéristiques qui se chevauchent, surtout au niveau des sépales. C'est pour cela que la silhouette de certaines observations de ces clusters est faible ou négative.

```
table(kmeans_clusters, iris$Species)
```

kmeans_clusters	setosa	versicolor	virginica
1	0	11	36
2	0	39	14
3	50	0	0